

End2Race: Efficient End-to-End Imitation Learning for Real-Time F1Tenth Racing

Anonymous Authors

Abstract—Autonomous racing serves as a compelling testbed for advancing autonomous driving systems. F1Tenth, a 1/10-scale autonomous racing platform, has been widely adopted for education and research. Annual competitions held worldwide bring teams together to compete in intense, head-to-head racing. Yet, leading solutions in these tournaments remain dominated by rule-based methods. While learning-based algorithms have been proposed, they focus primarily on single-vehicle settings, leaving multi-vehicle interactions largely unexplored. To address this gap, we introduce End2Race, an end-to-end learning framework specifically designed for head-to-head autonomous racing. Within this framework, we develop a computationally efficient recurrent policy network that achieves an inference time below 1 ms on an F1Tenth onboard computer. Extensive testing shows that the policy generalizes effectively across novel tracks, demonstrating high racing speeds, advanced overtaking capabilities, and robust safety. These results significantly surpass established rule-based baselines. The codebase will be made publicly available.

I. INTRODUCTION

Autonomous racing serves as a compelling proving ground for advancing autonomous vehicle systems. It combines perception, planning, and control under high-speed, interactive, and computationally constrained conditions. Enhancing algorithmic capabilities in this domain is directly relevant to the broader development of autonomous driving. F1Tenth [1], a 1/10-scale vehicular platform, provides an accessible testbed for autonomous racing education and research. Equipped with a 2D LiDAR or RGB-D camera, the vehicle can operate at up to 17.9 m/s. Each year, competitions are held worldwide and attract broad participation from the community.

Existing approaches for autonomous racing can be broadly grouped into map-based, reactive, and learning-based methods. Among them, map-based pipelines are the most widely used, relying on a pre-constructed map to follow a globally optimized raceline. Prior to racing, the track layout is mapped using SLAM [2]. A minimum-curvature raceline is then computed to serve as the global reference path [3]. During racing, a particle filter [4] tracks the vehicle’s position along the track, enabling a lattice planner [5] to evaluate feasible local trajectories. A motion controller, such as Pure Pursuit [6] or model predictive control (MPC) [7], then executes

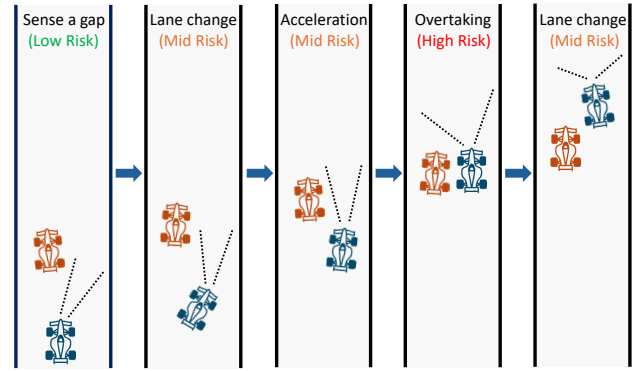


Fig. 1. Simple overtake scenario illustration.

the selected trajectory. However, map-based methods have two limitations. First, they require preprocessing and cannot operate on unseen tracks. Second, the multi-stage localization and planning pipeline incurs a delay of approximately 30 ms on onboard hardware [8], [9]. With additional sensor latency, the vehicle can travel up to a full vehicle length before updated control commands take effect. Although non-critical under most conditions, this delay becomes consequential when a high-speed straight leads directly into a sharp curve, leaving the vehicle susceptible to boundary collisions. Such collisions are widely observed in F1Tenth competitions.

Reactive methods provide a simpler, rule-based alternative by acting directly on current sensor measurements. Their map-free design avoids localization and planning, reducing computational cost while allowing operation on unknown tracks. The Follow-the-Gap algorithm [10], for example, identifies the largest free region and steers toward its midpoint. However, these methods operate instantaneously, as each decision is based on an isolated observation with limited temporal context, leading to planning inconsistencies and control instability [11].

Learning-based methods have also been widely proposed for F1Tenth. Early work focused primarily on single-vehicle navigation [12], [13], whereas more recent studies have shifted toward head-to-head competition. However, existing approaches either use only static obstacles during training before being tested directly against dynamic opponents [14], rely on a small set of human demonstrations and evaluation scenarios [15], or operate at low racing speeds [16].

In high-speed head-to-head competition, overtaking is a key determinant of the race outcome and presents an intrinsically challenging problem. Figure 1 illustrates a simplified five-stage overtaking process. First, the ego vehicle identifies

Funding information omitted for double-blind review.

¹Affiliation omitted for double-blind review.

²Affiliation omitted for double-blind review.

³Affiliation omitted for double-blind review.

³Affiliation omitted for double-blind review.

³Affiliation omitted for double-blind review.

³Affiliation omitted for double-blind review.

[†]Equal contribution.

*Corresponding author information omitted.

a suitable opening, executes a sharp lane change to align with it, and accelerates rapidly to catch up with the opponent. During the pass, the vehicles race side by side with minimal safety margins. After completing the pass, the ego vehicle returns to the track center to regain lateral clearance. Successful overtaking therefore depends on continuous decision-making, precise maneuver execution, rapid adaptation to the opponent's motion, and minimal sensing-to-control latency. This requires both sufficient model capacity and low inference latency, two conflicting demands that existing methods fail to reconcile.

To address these limitations, we propose **End2Race**, an end-to-end learning framework for real-time, head-to-head autonomous racing. The framework generates diverse overtaking scenarios across dynamic racing conditions, enabling systematic policy training and evaluation at scale. Utilizing this framework, we train a racing policy based on a Gated Recurrent Unit (GRU) architecture [17]. The policy takes a 360-degree 2D LiDAR scan and the current ego speed as inputs and directly outputs steering and speed commands, achieving an inference time below 1 ms on an F1Tenth onboard computer. Training follows a two-stage pipeline that initializes the policy through imitation learning and then fine-tunes it using reinforcement learning. Across both training and test tracks, the resulting policy demonstrates high racing speeds, strong safety performance, and competitive overtaking capabilities.

The main contributions of this work are as follows:

- 1) We introduce End2Race, an end-to-end learning framework for autonomous racing that supports scalable overtaking scenario generation, policy training, and benchmark evaluation.
- 2) We design a computationally efficient recurrent policy network with a sensing-to-control latency below 1 ms, supporting real-time operation at high racing speeds.
- 3) We show that our two-stage training pipeline combining imitation learning with reinforcement learning fine-tuning yields strong generalization across novel tracks, delivering safe and competitive racing behavior.

II. RELATED WORK

A. Imitation Learning

Imitation learning (IL) trains a driving policy from expert demonstrations. Sun et al. [12] generated demonstrations using a Pure Pursuit controller to evaluate BC [18], DAgger [19], HG-DAgger [20], and EIL [21], finding that all four policies completed the training track but failed to generalize to an unseen track. Paluch et al. [22] trained a feedforward policy to imitate nonlinear model predictive control along an offline raceline, achieving faster lap times than Pure Pursuit. TinyLidarNet [14] trained a compact convolutional network on static obstacle demonstrations and then evaluated it directly against dynamic opponents. Zhang and Loidl [15] trained dense and recurrent policies from human overtaking demonstrations, but relied on limited training data and evaluated across only a few scenarios.

B. Model-Free Reinforcement Learning

Model-free RL optimizes control policies through environmental interaction to maximize cumulative reward. Hamilton et al. [23] compared DDPG [24], SAC [25], and BC [18], finding that RL policies achieved faster lap times but incurred higher collision rates. Steiner et al. [16] trained a TD3 [26] policy for head-to-head racing, demonstrating that opponent-inclusive training improves overtaking success against diverse baselines. However, their deep feedforward policy network introduced substantial inference delays, compromising real-time responsiveness and forcing operating speeds below 2 m/s. Such velocities fall far below practical racing limits, leaving the challenges of high-speed multi-vehicle competition largely unaddressed.

C. Model-Based Reinforcement Learning

Model-based RL learns a predictive model of the environment to optimize control policies through imagined rollouts. For instance, Dreamer [27] learns a recurrent state-space model and trains actor-critic networks entirely within a latent space. Brunnbauer et al. [13] applied Dreamer to autonomous racing, outperforming model-free baselines in sample efficiency and lap completion.

D. Hybrid Learning Methods

Hybrid methods integrate classical planning or control modules into learned autonomy pipelines. Dwivedi et al. [28] augmented an RL tracking controller with reference trajectories from a global planner, combining low-level continuous control with high-level path guidance. Bhargav et al. [29] learned overtaking success probabilities across track segments to dynamically select discrete lateral planning modes. Similarly, Trumpp et al. [30] coupled residual reinforcement learning with an artificial potential field planner to support mapless multi-agent racing.

III. METHODOLOGY

We develop End2Race using the F1TENTH simulator, which provides high-fidelity vehicle dynamics and accurate 2D LiDAR simulation. Four tracks are selected reflecting real-world circuit layouts: Austin, Hockenheim, MoscowRaceway, and Nuerburgring. Among these, Austin is used for training, while the remaining three are reserved for evaluation. For each track, an automated workflow generates diverse overtaking scenarios, each pairing an ego vehicle with a leading opponent. Using demonstrations provided by a map-based expert, we first train a GRU network through Behavioral Cloning (BC) [18], and then fine-tune the resulting policy through Proximal Policy Optimization (PPO) [31]. The following sections describe each stage in detail (Fig. 2).

A. Overtaking Scenario Setup

Each overtaking scenario pairs an ego vehicle with an opponent initially positioned ahead of it and has a duration of 8 seconds. Vehicle placement is expressed in Frenet coordinates, in which the longitudinal direction measures progress around the circuit and the lateral direction specifies

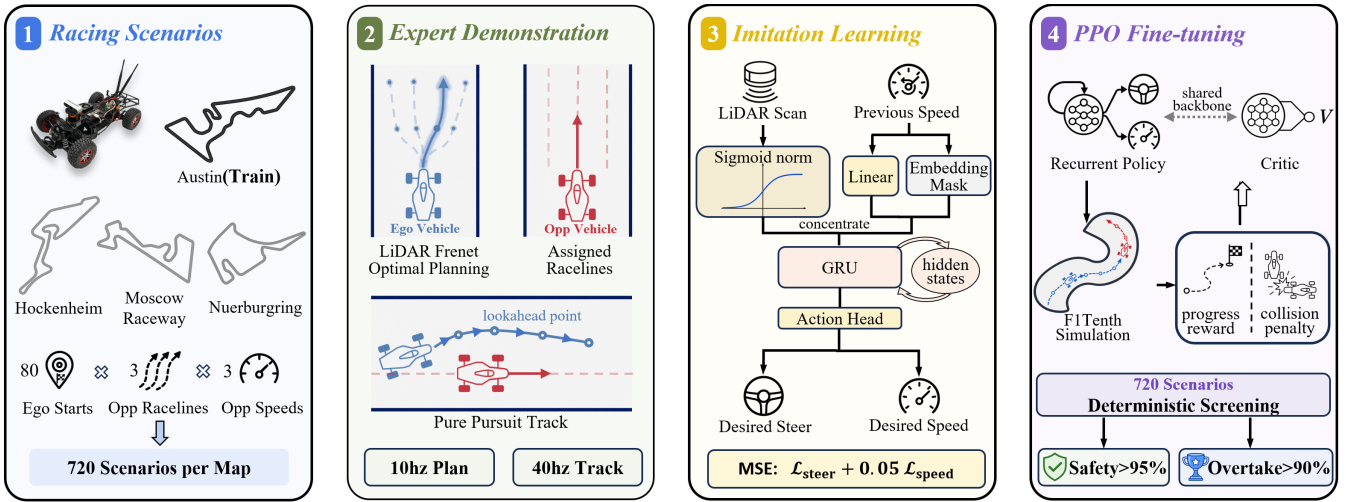


Fig. 2. End2Race system overview: (a) racing scenarios, (b) expert demonstration, (c) imitation learning, and (d) reinforcement learning.

position across the track width. We specify each scenario using four parameters, $(s_0, \Delta s_0, r_{\text{opp}}, \alpha_{\text{opp}})$.

1) *Ego Starting Position* (s_0): For each track, we define N_s ego starting positions equally spaced in the longitudinal direction, with $N_s = 80$ fixed across all tracks. For example, on the 420 m Austin track, adjacent starting positions are separated by approximately 5.2 m.

2) *Longitudinal Gap* (Δs_0): The longitudinal gap Δs_0 specifies how far the opponent is positioned ahead of the ego vehicle along the track at the start of a scenario. A small gap provides insufficient starting space for the ego vehicle to maneuver, while a large gap requires the ego vehicle to spend more time catching the opponent, leaving less time for interaction. We therefore set $\Delta s_0 = 3$ m to balance these considerations.

3) *Opponent Raceline* (r_{opp}): Beyond longitudinal placement, the lateral positions of the ego vehicle and opponent are critical to the overtaking geometry. We represent this lateral configuration using the opponent raceline r_{opp} , which assigns the opponent to one of three reference paths. Raceline generation follows the framework of Heilmeyer et al. [3], positioning left, center, and right paths at 35%, 50%, and 65% across the track width from the left boundary. On the Austin circuit, this yields approximately 0.25 m of lateral separation between the center and each side raceline. An opponent on the center raceline restricts lateral clearance on both sides and is harder to pass, whereas one on a side raceline leaves a wider passing corridor. Meanwhile, the ego vehicle always uses the center raceline as its reference, but may temporarily deviate from it as needed.

4) *Opponent Speed Scale* (α_{opp}): Each reference raceline provides an optimized speed profile based on track curvature and vehicle dynamics. Here, we set a maximum speed of 7.5 m/s, as empirical testing showed that higher values risk compromising tracking and control stability. Crucially, if the opponent follows this profile directly, the ego vehicle sharing the same profile cannot catch up or overtake. To create overtaking opportunities, we scale down the opponent's speed

by a factor of $\alpha_{\text{opp}} \in \{0.4, 0.6, 0.8\}$. Scales below 0.4 make the opponent unrealistically slow, while those above 0.8 often prevent the ego vehicle from catching up in time. Meanwhile, the ego vehicle always operates under the full speed limit.

Combining 80 ego starting positions s_0 , a fixed longitudinal gap Δs_0 , three opponent racelines r_{opp} , and three opponent speed scales α_{opp} yields 720 scenarios per track. During each 8-second scenario, the opponent passively tracks its assigned raceline with a Pure Pursuit controller. It does not react to the ego vehicle, nor does it actively engage in adversarial behavior, such as defensive blocking or erratic speed changes [33].

B. Expert Demonstration

We design a map-based lattice planner to generate overtaking demonstrations for the ego vehicle. The lattice planner, adapted from the Frenet Optimal Trajectory framework [34], converts each LiDAR scan into obstacle points and generates candidate trajectories across a range of lateral positions and terminal speeds. It then discards candidates that violate dynamic constraints and selects the trajectory τ that minimizes the cost $\mathcal{J}(\tau)$:

$$\mathcal{J}(\tau) = \frac{1}{N} \sum_{i=1}^N \left(1 - \frac{v_i}{v_{\text{max}}} \right) + \lambda_c \max_i \left[\max \left(0, 1 - \frac{d_i}{d_c} \right) \right]^2 \quad (1)$$

where N is the number of future trajectory points sampled at 0.1 s intervals ($N = 6$); v_i is the projected vehicle speed at point i , favoring trajectories with higher mean speeds; $v_{\text{max}} = 7.5$ m/s is the speed limit; d_i is the distance from point i to the nearest obstacle; $d_c = 0.5$ m is the clearance threshold for applying the collision penalty; and $\lambda_c = 5$ balances vehicle speed and obstacle clearance. The ego vehicle then tracks the selected trajectory using a Pure Pursuit controller.

C. Training Dataset

We collect 720 eight-second overtaking demonstrations on the Austin track. Training data are recorded at 40 Hz to



Fig. 3. Expert-planner failure during overtaking, shown in temporal order from (I) to (III). Blue denotes the ego vehicle and its planned trajectory, and red denotes the opponent. In this scenario, the opponent follows the right raceline with a speed scale of 0.8. (I) The ego approaches the opponent, preparing to overtake. (II) The vehicles travel side by side. (III) The ego returns to the center raceline prematurely, leading to a sideswipe collision.

reflect the onboard LiDAR-to-control update cycle, yielding 320 observation-action pairs per scenario. Among these scenarios, 620 (86.1%) result in successful overtakes, 23 (3.2%) in collision-free car-following, and 77 (10.7%) in collisions. These collisions stem primarily from the lattice planner’s forward-only obstacle evaluation, which does not account for a moving opponent alongside or behind the ego vehicle. Consequently, the planner may return to the center raceline before the ego vehicle has fully cleared the opponent. Collision risk increases as the speed difference between the two vehicles decreases, with most failures occurring at an opponent speed scale of 0.8. Figure 3 illustrates one such failure: during overtaking, the planned ego trajectory returns toward the center raceline while the vehicles still overlap longitudinally, resulting in a sideswipe collision.

These collisions can be mitigated by incorporating explicit opponent trajectory prediction and interaction modeling, as demonstrated in Predictive Spliner [35]. However, such approaches remain fundamentally constrained by hand-crafted heuristics and rigid safety margins that struggle to generalize across diverse head-to-head overtaking scenarios. We instead exclude the 77 collision scenarios from BC training and rely on downstream PPO fine-tuning (Sec. III-E) to learn safe interactive behavior.

D. Model Architecture

We use a GRU network that maps sequential observations directly to vehicle control commands. At each timestep, the observation comprises a 360-degree 2D LiDAR scan of 180 beams and the current ego speed, while the control output specifies the desired speed and steering angle.

1) *LiDAR Normalization*: Bufacchi and Iannetti show that human interaction with surrounding space is proximity-dependent, with behavioral responses strengthening progressively as object distance decreases [36]. The same principle can be applied to autonomous racing, where the influence of track boundaries and competing vehicles on ego motion similarly depends on their distance. To emulate this aspect of human spatial cognition, we transform each of the 180 LiDAR beams using a sigmoid function that maps the raw scan value, $x \in [0, 10]$, to a normalized value, $\sigma(x) \in [0, 1]$:

$$\sigma(x) = \left(-\frac{1}{1 + e^{-kx}} + 1 \right) \cdot 2 \quad (2)$$

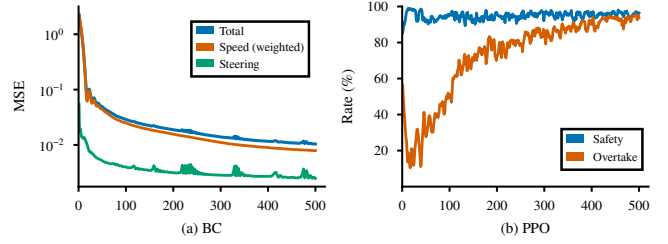


Fig. 4. Training curves for the two-stage policy optimization, with each stage trained for 500 epochs. (a) BC loss, with the speed loss weighted by 0.05. (b) PPO safety and overtake rates on the 720 Austin scenarios.

Here, the decay parameter k controls the effective sensing range and is set to 0.3. With this design, the normalized value increases sharply and nonlinearly as obstacles approach, while asymptotically decaying to zero with distance.

2) *Ego-Speed Embedding*: To maintain dynamic consistency, we incorporate the current ego speed into the policy. Specifically, the scalar speed v_t is projected through a linear layer into a 30-dimensional embedding $\psi(v_t)$. To prevent shortcut learning [37], in which the model simply copies the current speed into its speed prediction, we mask the projected speed with a learnable embedding $\mathbf{e}_{\text{mask}}$ with probability p :

$$\tilde{\psi}(v_t) = \begin{cases} \psi(v_t), & \text{with probability } 1 - p, \\ \mathbf{e}_{\text{mask}}, & \text{with probability } p, \end{cases} \quad (3)$$

Here, we set $p = 0.2$, encouraging the model to determine desired speed from LiDAR observations when the current speed is masked and to incorporate it when available.

3) *Policy Network*: The 180-dimensional normalized LiDAR scan and the 30-dimensional speed embedding are concatenated to form a 210-dimensional input vector. This input is passed to a single-layer GRU network, which maintains a 420-dimensional hidden state and updates it at each timestep. This hidden state is then passed to an MLP action head to produce the desired speed and steering angle. In total, the network has 850,556 trainable parameters.

E. Policy Training

Policy training consists of two stages: we first use BC to learn an initial policy from expert demonstrations, and then fine-tune it with PPO via direct environment interaction.

1) *Behavioral Cloning*: Each 8-second scenario is processed as a complete input sequence, generating a control prediction at every timestep and aggregating the losses across the full temporal horizon. The loss function combines speed and steering mean squared errors (MSE), weighting the speed loss by 0.05 to balance their scales:

$$\mathcal{L} = 0.05 \mathcal{L}_{\text{speed}} + \mathcal{L}_{\text{steer}}. \quad (4)$$

2) *Proximal Policy Optimization*: The policy is fine-tuned across the 720 training scenarios on the Austin track. During each rollout, the policy collects one trajectory per scenario, combining all 720 trajectories into a single batch to perform one policy update. At each timestep t , the reward function r_t combines a raceline-progress term with a collision penalty:

$$r_t = 0.01d_t - 3c_t, \quad (5)$$

TABLE I
BENCHMARK EVALUATION: SINGLE-VEHICLE AND HEAD-TO-HEAD RACING PERFORMANCE

Setting	Metric	Austin (Train)			Hockenheim			MoscowRaceway			Nuerburgring		
		Expert	BC	PPO	Expert	BC	PPO	Expert	BC	PPO	Expert	BC	PPO
Single-Vehicle	Mean Speed (m/s)	6.7	6.7	8.7	6.8	6.9	8.8	6.5	6.8	8.5	6.9	7.0	8.9
	Mean Lap Time (s)	63.2	62.7	48.7	53.2	52.5	41.1	50.6	47.7	38.5	64.7	63.7	50.2
Head-to-Head	Safety Rate (%)	89.3	82.8	98.1	91.8	78.6	97.5	87.2	76.7	97.1	92.8	81.5	97.9
	Overtake Rate (%)	86.1	62.8	96.1	90.6	66.1	97.1	83.5	69.0	96.8	92.4	74.6	97.5

where d_t denotes the distance traveled along the raceline during timestep t , encouraging higher vehicle speed, and c_t is a binary indicator equal to 1 if the ego vehicle collides and 0 otherwise. Together, these terms encourage the ego vehicle to prioritize safety while exploiting overtaking opportunities.

IV. EXPERIMENTS

We evaluate End2Race across the training track and three test tracks. The evaluation includes single-vehicle driving and head-to-head overtaking (Table I). Visual illustrations further demonstrate representative policy behaviors (Fig. 5).

A. Training Setup

The BC policy is trained for 500 epochs using the Adam optimizer at a learning rate of 10^{-4} . Training completes in less than five minutes on a single L40S GPU. The loss decreases rapidly during early epochs and then gradually converges (Fig. 4(a)). Additional training further reduces loss, but risks overfitting.

PPO starts from the trained BC checkpoint with a randomly initialized value head. Training runs for 500 epochs with a learning rate of 10^{-5} and a clipping range of $[0.8, 1.2]$. Since online simulation is time-consuming, training is accelerated with 64 CPU cores and four L40S GPUs, requiring approximately 16 hours in total. During early epochs, the safety rate rises sharply while the overtake rate drops, reflecting conservative driving. As training progresses, the overtake rate steadily increases while maintaining high safety, indicating that the policy learns to execute overtaking maneuvers without compromising collision avoidance (Fig. 4(b)).

B. Single-Vehicle Driving

We first evaluate End2Race in a single-vehicle setting to test whether the policy can complete the track at high speeds. Table I reports the mean speed and lap time of three methods: the map-based expert, BC policy, and PPO policy, all of which complete the evaluation without collision. Compared to the expert, which relies on explicit track knowledge and an optimized reference raceline, the map-free BC policy achieves comparable mean speeds and lap times across all four tracks, demonstrating effective learning and generalization.

The PPO policy achieves higher mean speeds and shorter lap times on every track. These gains arise from continued interactive exploration, through which the policy discovers racing strategies that are often more competitive than those

demonstrated by the expert and closely resemble techniques used by human racers. One example is corner-exit track-out, in which the vehicle moves from the inside to the outside of a curve to reduce speed loss. Another example is corner-exit acceleration and corner-entry braking, in which the vehicle accelerates immediately upon exiting a curve, maintains high speed along the following straight, and brakes sharply just before entering the next curve [38].

C. Head-to-Head Racing

We evaluate head-to-head racing performance using the overtake and safety rates. The overtake rate is defined as the proportion of scenarios ending in a successful overtake, while the safety rate includes both successful overtakes and collision-free car-following outcomes. The map-based expert achieves a safety rate of approximately 90% across all tracks, with the vast majority being successful overtakes.

Compared to the expert, the BC policy adopts a more conservative racing strategy, achieving lower overtake rates. However, this conservatism does not translate into improved safety. We attribute this outcome to the imitation learning objective itself: while the network mimics demonstrated control commands, supervised action matching does not establish the internal representations and tactical reasoning essential for vehicle interactions. Consequently, the policy remains effective for single-vehicle driving but struggles in more complex head-to-head scenarios.

This limitation is resolved during the PPO fine-tuning stage, which bridges the performance gap and even exceeds the expert, achieving higher safety and overtake rates across all tracks. This demonstrates that PPO not only develops adaptive interactive behaviors, but also overcomes the limitations of the expert demonstration data.

D. Scenario Illustration

Figure 5 illustrates four head-to-head scenarios that highlight the model’s behavior during overtaking encounters. All episodes evaluate the PPO policy on the Austin circuit.

Overtaking: Fig. 5(a) illustrates a nominal overtaking sequence along a straight segment. Upon observing the slower opponent ahead, the ego vehicle initiates a lateral shift while sustaining cruising speed. Alongside the opponent, the ego vehicle maintains a stable lateral margin throughout the pass. Once past the opponent, the ego vehicle smoothly returns to the track center, avoiding an abrupt cut-back.

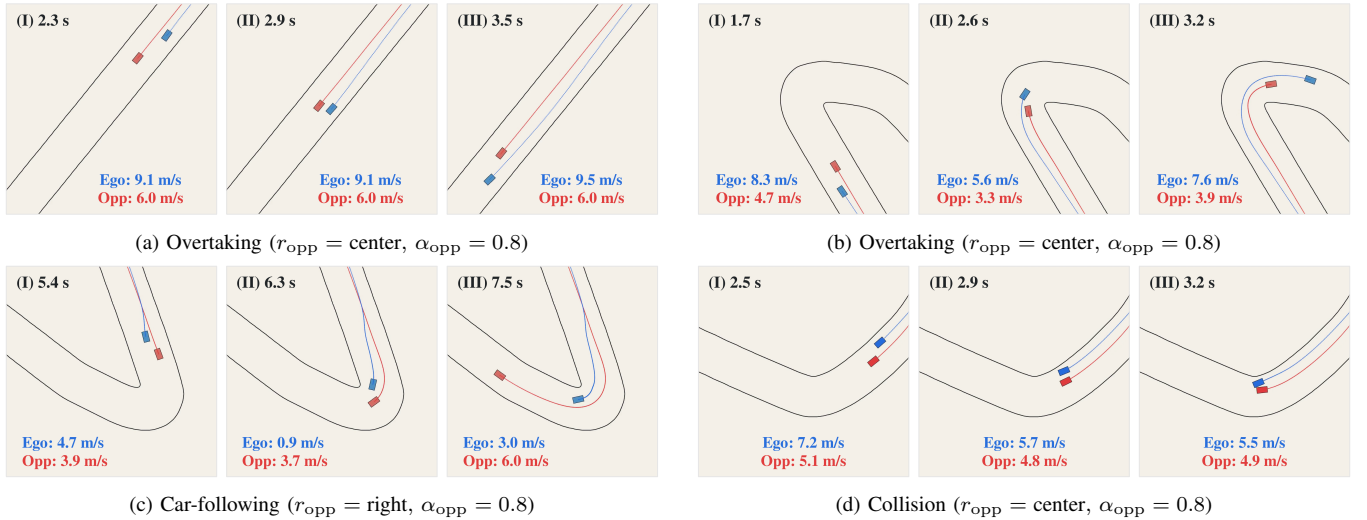


Fig. 5. Representative head-to-head interaction scenarios across time on the Austin circuit. Blue denotes the ego vehicle and red denotes the opponent, with historical trajectories rendered in matching colors. Each frame indicates the elapsed time and vehicle speeds.

Fig. 5(b) depicts a pass through a sharp curve. The ego vehicle follows the outside line behind the slower opponent, preparing the pass on approach to the turn. Entering the curve, it maintains this line under moderate braking, leveraging its speed advantage to execute the maneuver. Early acceleration before corner exit builds speed to pull ahead of the opponent and complete the pass.

Car-Following: Fig. 5(c) depicts car-following behavior under spatial constraints. The ego vehicle initially prepares an overtake behind the opponent. However, upon entering the curve, the opponent completely blocks the inner corridor, forcing the ego vehicle to brake heavily and almost come to a complete stop. After the opponent advances through the curve, the ego vehicle accelerates to resume speed.

Collision: Fig. 5(d) depicts a collision case. The ego vehicle prepares to overtake behind the opponent, and the vehicles travel side by side as the track suddenly narrows. Left with no room along the boundary side, the ego vehicle has no choice but to steer toward the opponent, resulting in a collision.

E. Ablation Studies

We conduct an ablation study to examine the effect of different design choices in the model architecture (Table II). The evaluation covers four variants across LiDAR normalization and temporal sequence modeling. All models are trained under identical behavior cloning settings and evaluated on the Austin circuit.

1) *LiDAR Normalization:* We compare the proposed sigmoid normalization with two alternatives: no normalization and linear normalization, all sharing the same GRU architecture. The unnormalized model fails to complete a single-vehicle lap. Linear normalization provides a viable policy, but leaves a large performance gap in head-to-head racing, with marked drops in both overtake and safety rates. This comparison reveals that the sigmoid formulation uniquely provides the proximity representation required for safe and effective racing.

TABLE II
ABLATION STUDY ON THE AUSTIN CIRCUIT

Variant	Single-Vehicle		Head-to-Head	
	Speed (m/s)	Lap Time (s)	Overtake (%)	Safety (%)
No Norm.	—	—	16.3	21.1
Linear Norm.	6.4	64.9	47.0	73.3
MLP	6.9	60.9	67.6	70.4
Transformer	6.6	63.8	73.1	80.6
GRU (BC)	6.7	62.7	62.8	82.8

2) *Temporal Information:* To assess the importance of temporal information, we evaluate an MLP that operates only on the latest observation. The MLP replaces the recurrent unit with a $210 \rightarrow 420$ linear layer, followed by the action head. The resulting policy exhibits heuristic driving behavior, achieving a higher vehicle speed and overtake rate at the cost of a substantial drop in safety. This demonstrates that temporal cues are essential for continuous interaction modeling and adaptive racing behavior.

3) *Temporal Modeling:* We evaluate a Transformer [39] as an alternative temporal model. The model operates on a maximum history sequence of 20 tokens, which corresponds to a span of 0.5 s. Each token represents one timestep and has a dimension of 210. The tokens are processed through two encoder layers with bidirectional self-attention. The number of attention heads is set to three, and the feedforward network has a hidden dimension of 420. The output of the latest token feeds into the action head to produce the control commands. The resulting policy achieves a higher overtake rate than the baseline, but with a slightly lower safety rate. Additionally, the Transformer model incurs *four times the inference latency* of the GRU. To balance safety, overtaking, and real-time responsiveness, we retain the GRU as our default architecture. Meanwhile, we recognize the Transformer as a practical alternative.

V. CONCLUSION

In this work, we introduce **End2Race**, an end-to-end learning framework for autonomous racing that supports scalable overtaking scenario generation, policy training, and benchmark evaluation. Our proposed policy achieves low inference latency and delivers competitive racing performance across novel tracks. Future work will focus on deploying the policy to a physical vehicle and participating in F1Tenth competitions.

REFERENCES

- [1] M. O’Kelly, H. Zheng, D. Karthik, and R. Mangharam, “F1Tenth: An open-source evaluation environment for continuous control and reinforcement learning,” *Proceedings of Machine Learning Research*, vol. 123, 2020.
- [2] S. Macenski and I. Jambrecic, “Slam toolbox: Slam for the dynamic world,” *Journal of Open Source Software*, vol. 6, no. 61, p. 2783, 2021.
- [3] A. Heilmeier, A. Wischniewski, L. Hermansdorfer, J. Betz, M. Lienkamp, and B. Lohmann, “Minimum curvature trajectory planning and control for an autonomous race car,” *Vehicle System Dynamics*, vol. 58, no. 10, pp. 1497–1527, 2020.
- [4] C. Walsh and S. Karaman, “Cddt: Fast approximate 2d ray casting for accelerated localization,” *arXiv preprint arXiv:1705.01167*, 2017.
- [5] M. Pivtoraiko, R. A. Knepper, and A. Kelly, “Differentially constrained mobile robot motion planning in state lattices,” *Journal of Field Robotics*, vol. 26, pp. 308 – 333, March 2009.
- [6] R. C. Coulter, “Implementation of the pure pursuit path tracking algorithm,” Tech. Rep. CMU-RI-TR-92-01, Carnegie Mellon University, Pittsburgh, PA, January 1992.
- [7] P. Falcone, F. Borrelli, J. Asgari, H. E. Tseng, and D. Hrovat, “Predictive active steering control for autonomous vehicle systems,” *IEEE Transactions on Control Systems Technology*, vol. 15, no. 3, pp. 566–580, 2007.
- [8] Z. Zang, H. Zheng, J. Betz, and R. Mangharam, “Local-INN: Implicit map representation and localization with invertible neural networks,” in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 11742–11748, 2023.
- [9] F. Muzzini, N. Capodiceci, F. Ramanzin, and P. Burgio, “Gpu implementation of the frenet path planner for embedded autonomous systems: A case study in the F1TENTH scenario,” *Journal of Systems Architecture*, vol. 154, p. 103239, 2024.
- [10] V. Sezer and M. Gokasan, “A novel obstacle avoidance algorithm: “follow the gap method,”” *Robotics and Autonomous Systems*, vol. 60, no. 9, pp. 1123–1134, 2012.
- [11] T. Hossain, H. Habibullah, R. Islam, and R. V. Padilla, “Local path planning for autonomous mobile robots by integrating modified dynamic-window approach and improved follow the gap method,” *Journal of Field Robotics*, vol. 39, no. 4, pp. 371–386, 2022.
- [12] X. Sun, M. Zhou, Z. Zhuang, S. Yang, J. Betz, and R. Mangharam, “A benchmark comparison of imitation learning-based control policies for autonomous racing,” in *2023 IEEE Intelligent Vehicles Symposium (IV)*, pp. 1–5, 2023.
- [13] A. Brunnbauer, L. Berducci, A. Brandstätter, M. Lechner, R. Hasani, D. Rus, and R. Grosu, “Latent imagination facilitates zero-shot transfer in autonomous racing,” in *2022 International Conference on Robotics and Automation (ICRA)*, pp. 7513–7520, 2022.
- [14] M. M. Zarrar, Q. Weng, B. Yerjan, A. Soyuyigit, and H. Yun, “Tinylidarnet: 2d lidar-based end-to-end deep learning model for f1tenth autonomous racing,” 2024.
- [15] J. Zhang and H. Loidl, “F1Tenth: An over-taking algorithm using machine learning,” in *2023 28th International Conference on Automation and Computing (ICAC)*, pp. 01–06, 2023.
- [16] E. Steiner, D. van der Spuy, F. Zhou, A. Pama, M. Liarokapis, and H. Williams, “Accelerating real-world overtaking in f1tenth racing employing reinforcement learning methods,” in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4528–4534, IEEE, 2025.
- [17] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, “Learning phrase representations using rnn encoder-decoder for statistical machine translation,” *arXiv preprint arXiv:1406.1078*, 2014.
- [18] C. Sammut, “Behavioral cloning,” 01 2011.
- [19] S. Ross, G. Gordon, and D. Bagnell, “A reduction of imitation learning and structured prediction to no-regret online learning,” in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*, pp. 627–635, JMLR Workshop and Conference Proceedings, 2011.
- [20] M. Kelly, C. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, “Hg-dagger: Interactive imitation learning with human experts,” in *2019 International Conference on Robotics and Automation (ICRA)*, pp. 8077–8083, IEEE, 2019.
- [21] J. Spencer, S. Choudhury, M. Barnes, M. Schmittle, M. Chiang, P. Ramadge, and S. Srinivasa, “Expert intervention learning: An online framework for robot learning from explicit and implicit human feedback,” *Autonomous Robots*, vol. 46, no. 1, pp. 99–113, 2022.
- [22] M. Paluch, F. Bolli, X. Deng, A. Rios Navarro, C. Gao, and T. Delbruck, “Fpga hardware neural control of cartpole and f1tenth race car,” 2024.
- [23] N. Hamilton, P. Musau, D. M. Lopez, and T. T. Johnson, “Zero-shot policy transfer in autonomous racing: Reinforcement learning vs imitation learning,” in *2022 IEEE International Conference on Assured Autonomy (ICAA)*, pp. 11–20, 2022.
- [24] T. P. Lillicrap, J. J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, and D. Wierstra, “Continuous control with deep reinforcement learning,” in *International Conference on Learning Representations*, 2016.
- [25] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *International conference on machine learning*, pp. 1861–1870, Pmlr, 2018.
- [26] S. Fujimoto, H. Hoof, and D. Meger, “Addressing function approximation error in actor-critic methods,” in *International conference on machine learning*, pp. 1587–1596, PMLR, 2018.
- [27] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, “Dream to control: Learning behaviors by latent imagination,” *arXiv preprint arXiv:1912.01603*, 2019.
- [28] T. Dwivedi, T. Betz, F. Sauerbeck, P. Manivannan, and M. Lienkamp, “Continuous control of autonomous vehicles using plan-assisted deep reinforcement learning,” in *2022 22nd International Conference on Control, Automation and Systems (ICCAS)*, pp. 244–250, 2022.
- [29] J. Bhargav, J. Betz, H. Zheng, and R. Mangharam, “Track based offline policy learning for overtaking maneuvers with autonomous racecars,” 2021.
- [30] R. Trumpp, E. Javanmardi, J. Nakazato, M. Tsukada, and M. Caccamo, “Racemop: Mapless online path planning for multi-agent autonomous racing using residual policy learning,” in *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 8449–8456, 2024.
- [31] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017.
- [32] M. Werling, J. Ziegler, S. Kammel, and S. Thrun, “Optimal trajectory generation for dynamic street scenarios in a frenet frame,” in *2010 IEEE International Conference on Robotics and Automation*, pp. 987–993, 2010.
- [33] H. Zheng, Z. Zhuang, J. Betz, and R. Mangharam, “Game-theoretic objective space planning,” 2023.
- [34] A. Sakai, D. Ingram, J. Dinius, K. Chawla, A. Raffin, and A. Pagues, “PythonRobotics: A python code collection of robotics algorithms,” 2018.
- [35] N. Baumann, E. Ghignone, C. Hu, B. Hildisch, T. Hämmerle, A. Bettoni, A. Carron, L. Xie, and M. Magno, “Predictive spliner: Data-driven overtaking in autonomous racing using opponent trajectory prediction,” *IEEE Robotics and Automation Letters*, vol. 10, no. 2, pp. 1816–1823, 2025.
- [36] R. J. Bufoacchi and G. D. Iannetti, “An action field theory of peripersonal space,” *Trends in Cognitive Sciences*, vol. 22, no. 12, pp. 1076–1090, 2018.
- [37] R. Geirhos, J. Jörn-Henrik, C. Michaelis, R. Zemel, B. Wieland, B. Matthias, and F. A. Wichmann, “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 11 2020.
- [38] W. C. Mitchell, R. Schroer, and D. B. Grisez, “Driving the traction circle,” Tech. Rep. 2004-01-3545, SAE International, 2004.
- [39] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017.